Kode Program Pengembangan Model

1. Import Library

```
!pip install micromlgen
# Library
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Preprocessing data
from sklearn.model_selection import train_test_split

# Random Forest
from sklearn.ensemble import RandomForestClassifier

# Pengujian Metrik
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report, roc_auc_score, roc_curve

# Micromlgen
import micromlgen

# Google drive
from google.colab import drive
```

2. Preprocessing Data

```
# Ambil dataset melalui URL
```

```
data_url =
"https://raw.githubusercontent.com/blacksonatae/Smart_Irigation_Dataset/main/Soil%20Moisture%2C%20Air%20Temperature%20and%20humidity%2C%20and%20Water%20Motor%20onoff%20Monitor%20data.AmritpalKaur.csv"

# Membaca dan menyimpan dataset ke dalam variable dalam bentuk data frame
data_irigation = pd.read_csv(data_url)

# Tampilkan 10 baris data
print(data_irigation.head(10))

# 1. Periksa missing value pada dataset irigasi
missing_values = data_irigation.isnull().sum()

# Tampilkan jumlah indikasi missing value pada fitur data
print("Jumlah sampel yang terindikasi missing value\npada fitur data:\n")
print(missing_values.to_frame(name="Jumlah missing")
    .rename_axis("Fitur Data")
    .reset_index()
    .to_string(index=False))

# 2. Memisahkan dataset menjadi fitur dan target
X = data_irigation[['Soil Moisture', 'Temperature', 'Air Humidity']]
y = data_irigation['Pump Data']

# 3. Membagi dataset menjadi data training dan testing
# train_size = 0.8 dan test_size=0.2
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.8,
test_size=0.2, random_state=42)

# Menampilkan jumlah sampel untuk training dan testing data
```

```
print("x_train shape (features): ", X_train.shape)
print("y_train shape (targets): ", y_train.shape)
print("X_test shape (features): ", X_test.shape)
print("y_test shape (targets): ", y_test.shape)

# 3. Buat boxplot untuk mendeteksi outlier secara visual
fig, axes = plt.subplots(1, 3, figsize=(10,4))

# Boxplot untuk fitur Soil Moisture
sns.boxplot(data=X_train, y='Soil Moisture', ax=axes[0], color="skyblue")
axes[0].set_title('Soil Moisture Outliers')

# Boxplot untuk fitur Temperature
sns.boxplot(data=X_train, y='Temperature', ax=axes[1], color="orange")
axes[1].set_title('Temperature Outliers')

# Boxplot untuk fitur Soil Moisture
sns.boxplot(data=X_train, y='Air Humidity', ax=axes[2], color="green")
axes[2].set_title('Air Humidity Outliers')

plt.tight_layout()
plt.show()

# 3.1 Hitung IQR untuk mendeteksi outlier
Q1 = X_train.quantile(0.25)
Q3 = X_train.quantile(0.75)
IQR = Q3 - Q1

# Menghitung jumlah outlier per fitur
outliers = ((X_train < (Q1 - 1.5 * IQR)) | (X_train > (Q3 + 1.5 * IQR))).sum()
```

```
# Buat hasil outlier dalam bentuk tabel
outliers_table = pd.DataFrame({'Fitur': outliers.index, 'Jumlah Outlier' :
outliers.values})

outliers_table = outliers_table.reset_index(drop=True)

print(outliers_table.to_string(index=False))

# 4. Lakukan pemeriksaan keseimbangan data

# Hitung jumlah dan persentase
class_distribution = y_train.value_counts()
class_percentages = (class_distribution / len(y_train)) * 100

# Buat laporan dalam bentuk tabel
class_distribution_table = pd.DataFrame({
    "Pump Data": class_distribution.index,
    "Jumlah": class_distribution.values,
    "Persentase %": class_percentages.values
})

# Rapikan angka desimal
class_distribution_table["Persentase %"] = class_distribution_table["Persentase
%"].round(2)

# Urutkan kelas
class_distribution_table = class_distribution_table.sort_values(by="Pump Data")

class_distribution_table = class_distribution_table.reset_index(drop=True)

print(class_distribution_table.to_string(index=False))
```

```
# Tampilkan visualisasi distribusi kelas melalui plot
plt.figure(figsize=(5, 5))
sns.barplot(x=class_distribution.index, y=class_distribution.values, width=0.5)
plt.title("Distribusi Kelas Target (Pump Data)", fontsize=12)
plt.xlabel("Kelas Target", fontsize=14)
plt.ylabel("Jumlah Sampel", fontsize=14)
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)

plt.show()

# 5 Kategorisasi data
# Fungsi Kategorisasi Kelembaban Tanah
def categorize_soil_moisture(moisture):
    if moisture >= 727:
        return 'Sangat Kering'
    elif moisture >= 618:
        return 'Kering'
    elif moisture >= 508:
        return 'Normal'
    elif moisture >= 398:
        return 'Basah'
    else:
        return 'Sangat Basah'
# Fungsi Kategorisasi Suhu Udara
def categorize_temperature(temperature):
    if temperature <= 23:
        return 'Dingin'
    elif 24 <= temperature <= 30:
        return 'Normal'
```

```
    else:
        return 'Panas'
# Fungsi Kategorisasi Kelembaban Udara
def categorize_air_humidity(humidity):
    if humidity <= 55:
        return 'Kering'
    else:
        return 'Lembab'
# Fungsi Kategorisasi Pump Data
def categorize_pump_data(pump_data):
    if pump_data == 0:
        return 'Hidup'
    else:
        return 'Mati'

soil_moisture = X_train['Soil Moisture']
soil_moisture_category = soil_moisture.apply(categorize_soil_moisture)

temperature = X_train['Temperature']
temperature_category = temperature.apply(categorize_temperature)

air_humidity = X_train['Air Humidity']
air_humidity_category = air_humidity.apply(categorize_air_humidity)

pump_data = y_train
pump_data_category = pump_data.apply(categorize_pump_data)

# Mensatukan soil_moisture dan soil_moisture_category
soil_moisture_df = pd.DataFrame({
    'Soil Moisture': soil_moisture,
    'Soil Moisture Category': soil_moisture_category
```

```
})
# Mensatukan temperature dan temperature_category
temperature_df = pd.DataFrame({
    'Temperature': temperature,
    'Temperature Category': temperature_category
})
# Mensatukan air_humidity dan air_humidity_category
air_humidity_df = pd.DataFrame({
    'Air Humidity': air_humidity,
    'Air Humidity Category': air_humidity_category
})
# Mensatukan pump_data dan pump_data_category
pump_data_df = pd.DataFrame({
    'Pump Data': pump_data,
    'Pump Data Category': pump_data_category
})
# Menggabungkan seluruh DataFrame menjadi satu untuk analisis
combined_df = pd.concat([soil_moisture_df, temperature_df, air_humidity_df,
pump_data_df], axis=1)
# Menampilkan DataFrame hasil gabungan
pd.set_option('display.max_rows', None)  # Menampilkan semua baris
pd.set_option('display.max_columns', None)  # Menampilkan semua kolom

combined_df = combined_df.reset_index(drop=True)

combined_df.head(100)  # Menampilkan 10 baris pertama

**3. Implementasi Random Forest**

# Random Forest Classifier
```

```
forest = RandomForestClassifier(n_estimators=100, max_features= "sqrt",
criterion="gini", bootstrap=True, max_depth=6, random_state=42)

# Training model
forest.fit(X_train, y_train)
```

4. Pengujian Random Forest

```
# 1. Pengujian Metrik
y_train_pred = forest.predict(X_train)
train_accuracy = accuracy_score(y_train, y_train_pred)

# Hitung akurasi testing
y_test_pred = forest.predict(X_test)
test_accuracy = accuracy_score(y_test, y_test_pred)

# Menampilkan nilai perameter max_depth
print(f"Max_depth: {forest.max_depth}")

# Tampilkan hasil evaluasi akurasi
print(f"Training Accuracy: {train_accuracy*100:.2f}%")
print(f"Test Accuracy: {test_accuracy*100:.2f}%")

# Tampilkan classification report

print("\nClassification Report:")
print(classification_report(y_test, y_test_pred, target_names= ["ON", "OFF"],
digits=4))

# Hitung confusion matrix
cm = confusion_matrix(y_test, y_test_pred)
```

```
# Tampilkan Confusion Matrix
plt.figure(figsize=(3, 3))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,
xticklabels=['ON', 'OFF'],yticklabels=['ON', 'OFF'])
plt.title('Confusion Matrix')
plt.ylabel('True label')
plt.xlabel('Predicted label')

plt.figtext(0.5, -0.15, f"Max Depth: {forest.max_depth}", ha="center",
fontsize=10)

plt.show()

# Prediksi probabilitas pada data test
# Ambil probabilitas kelas positif
y_test_proba = forest.predict_proba(X_test)[:, 1]

# Hitung AUC score sebagai metrik evaluasi
auc_score = roc_auc_score(y_test, y_test_proba)
print(f"AUC Score of Max Depth {forest.max_depth}: {auc_score}")

# Hitung nilai ROC curve (FPR & TPR)
fpr, tpr, thresholds = roc_curve(y_test, y_test_proba)

# Visualisasi
plt.figure(figsize=(5, 3))
plt.plot(fpr, tpr, color='orange',
        label=f'ROC Curve (AUC = {auc_score*100:.4f})')
plt.plot([0, 1], [0, 1], color='black', linestyle='--', label='Random Forest
Classifier')
```

```
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend(loc='lower right')

plt.figtext(0.5, -0.15, f"Max Depth: {forest.max_depth}",
            ha="center", fontsize=10)

plt.show()

# Konversi Random Forest ke C++ menggunakan micromlgen
code = micromlgen.port(forest)

# Buat file
filename = f"random_forest_classifier_m_{forest.max_depth}.h"

# Simpan file
with open(filename, "w") as f:
  f.write(code)

print(f"Konversi berhasil\nFile:{filename}\nMax Depth:{forest.max_depth}")
```